Study Notes: Days 50–56 — Advanced React + Full Stack

These notes follow Month 2, Week 4 of the roadmap, which is **Week 8 overall**. React snippets assume a React + TypeScript project. The full-stack feature is an illustrative local exercise, not a claim about Sai's production work.

Day 50 — useMemo / useCallback / React.memo

Month 2 — Full-Stack Depth, Week 4 — Advanced React + Full Stack (Week 8 overall).

Goal: Understand what each memoization mechanism caches, when stable identity matters, and why optimization must be measured.

`useMemo`

`useMemo` caches the result of a calculation between renders while its dependencies compare equal using `Object.is`. It can avoid repeating an expensive pure calculation or preserve a value's reference identity for a downstream optimization. The calculation runs during rendering, so it must be pure and must include every reactive value it reads in the dependency list.

Memoization has costs: dependency comparisons, retained memory, and added complexity. A cheap calculation may be clearer and just as fast without it. React can discard a memoized value in certain circumstances, so do not rely on `useMemo` for correctness, resource ownership, or a permanent singleton. Where the React Compiler is enabled and applicable, it can perform some memoization automatically; profile the actual application rather than assuming either manual or automatic memoization is always needed.

`useCallback`

`useCallback` caches a function reference while its dependencies remain equal. It is useful when that reference is passed to a memoized child or used as a dependency where stable identity matters. It does not prevent JavaScript from evaluating the function expression passed to the hook, make the function execute faster, or automatically prevent a component from rendering.

The callback closes over values from a render. Missing dependencies can leave it using stale state or props. A functional state updater can sometimes remove a dependency on the current state when the next state genuinely depends only on the previous value.

`React.memo`

`React.memo` lets React usually skip re-rendering a component when its props are unchanged; by default, individual props are compared with `Object.is`. It is a performance optimization, not a guarantee. A memoized component still updates for its own state or consumed context, and React may render it when necessary.

New object, array, or function references can defeat the default comparison even when their contents look identical. Conversely, mutating an existing object can hide a meaningful change. Custom comparison functions add cost and must compare all behaviorally relevant props, including callbacks; ignoring a changed

callback can preserve a stale closure. Prefer simple component boundaries and measurements before adding custom comparisons.

Code example

```
import { memo, useCallback, useMemo, useState } from "react";

type Item = { id: string; name: string };

const ResultList = memo(function ResultList({
  items, onSelect,
}: {
  items: Item[];
  onSelect: (id: string) => void;
}) {
  return (
    <ul>
      {items.map(item => (
        <li key={item.id}>
          <button type="button" onClick={() => onSelect(item.id)}>{item.name}
</button>
        </li>
      ))}
    </ul>
  );
});

export function Search({
  items, onSelect,
}: {
  items: Item[];
  onSelect: (id: string) => void;
}) {
  const [query, setQuery] = useState("");
  const visible = useMemo(
    () => items.filter(item =>
item.name.toLowerCase().includes(query.toLowerCase())),
    [items, query],
  );
  const selectItem = useCallback((id: string) => onSelect(id), [onSelect]);

  return (
    <>
      <label>
        Search
        <input value={query} onChange={event => setQuery(event.target.value)} />
      </label>
      <ResultList items={visible} onSelect={selectItem} />
    </>
  );
}
```

This only helps skip `ResultList` work when its `items` and `onSelect` references remain stable. If the parent recreates `items` or `onSelect` on every render, these dependencies change. The filter shown may be too cheap to justify memoization; measure before using this pattern.

DSA problem: Detect a cycle in a linked list

Problem: Return whether a singly linked list contains a cycle, without modifying it.

Approach: Floyd's tortoise-and-hare algorithm advances one pointer by one link and another by two. If a cycle exists, the faster pointer eventually catches the slower one inside it; if the fast pointer reaches the end, there is no cycle.

Complexity: O(n) time and O(1) additional space, where `n` is the number of distinct nodes reachable from the head.

```
class ListNode {
  constructor(public value: number, public next: ListNode | null = null) {}
}

function hasCycle(head: ListNode | null): boolean {
  let slow = head;
  let fast = head;
  while (slow !== null && fast !== null && fast.next !== null) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow === fast) return true;
  }
  return false;
}

const first = new ListNode(1);
const second = new ListNode(2);
first.next = second;
second.next = first;
console.log(hasCycle(first)); // true
console.log(hasCycle(new ListNode(3))); // false
```

Edge cases: Empty and single-node acyclic lists return false; a self-loop returns true. Compare node references, not node values: repeated values do not imply a cycle.

Likely interview questions

1. What does `useMemo` cache, and when may React discard it?
2. How is `useCallback` different from `useMemo`?
3. What can cause a memoized child to render despite unchanged parent props?
4. Why can a new array or callback defeat `React.memo`?
5. How can an incorrect dependency list create a stale closure?
6. What risks come with a custom prop comparison?
7. How would you prove memoization improves a real interaction?

8. Why does Floyd's algorithm compare node references rather than values?

Common pitfalls / misconceptions

- Memoizing every value without measuring the cost or benefit.
- Using memoization as a correctness guarantee.
- Omitting dependencies or ignoring changed callbacks in custom comparisons.
- Treating duplicate node values as a linked-list cycle.

Cheat-sheet recap

- `useMemo`: cached calculation result; `useCallback`: cached function reference.
- `React.memo`: usually skips parent-driven renders when props compare equal.
- State, context, dependencies, and reference identity still matter.
- Floyd's slow/fast pointers detect a cycle in O(n) time and O(1) space.

Day 51 — Re-rendering behavior

Month 2 — Full-Stack Depth, Week 4 — Advanced React + Full Stack (Week 8 overall).

Goal: Identify why React renders components, distinguish rendering from DOM updates, and locate unnecessary work without breaking correctness.

What causes renders

State updates, changed props from a parent, and changed context values can cause a component to render. When a parent renders, React normally visits its children as part of reconciling the subtree; a memoized child may be skipped if its props are unchanged, but its own state or consumed context can still update it. A render means React calls component code to calculate UI; it does not necessarily mean the DOM changes.

React can batch updates and may interrupt, restart, or discard render work depending on the situation. Render code should be pure. Development Strict Mode intentionally performs extra checks, so console counts in development do not directly represent production performance.

Identity, state, and context

React associates state with a component's position, type, and key in the tree. Changing a key or replacing a component type resets the corresponding state. Defining a new component function inside another component can create a new component type on each render and unexpectedly remount the subtree. Stable keys identify siblings, not global entities.

Passing fresh object or function props changes their identity; this matters for memoized children and effect dependencies, not because every new object is automatically expensive. Context consumers update when the provider supplies a changed value according to React's comparison, even if they are wrapped in `memo`. Splitting contexts by update frequency or moving state closer to consumers can reduce fan-out when measurements show a problem.

Investigating renders

Use React DevTools and the Profiler to observe which components render and how long commits take. Profile realistic interactions and data sizes using a suitable production or profiling build; development-only checks can distort timings. A frequent render is not necessarily a problem if it is cheap, and a single expensive render can matter more than many cheap ones.

First remove unnecessary state or effects, colocate state, and simplify expensive work. Then consider memoization or virtualization if the measured bottleneck warrants it. Check user-visible correctness after any change: an optimization that prevents a needed update is a bug.

Code example

```tsx
import { memo, useState } from "react";

const Greeting = memo(function Greeting({ name }: { name: string }) {
  return <p>Hello, {name}</p>;
});

export function Dashboard() {
  const [count, setCount] = useState(0);
  return (
    <>
      <button type="button" onClick={() => setCount(value => value + 1)}>
        Count: {count}
      </button>
      <Greeting name="Sai" />
    </>
  );
}
```

Updating `count` renders `Dashboard`. Because `Greeting` receives the same primitive prop, React can skip its parent-driven render. This example does not prove a measurable performance improvement; `Greeting` could still render for its own state or consumed context.

Likely interview questions

1. What is the difference between a component render and a DOM mutation?
2. Why can a child render when its parent renders even if its displayed data is unchanged?
3. What can update a memoized component?
4. How do keys and component types affect state preservation?
5. Why can defining a component inside another component cause remounts?
6. How do context values affect consumers?
7. Why can development Strict Mode make render counts misleading?
8. How would you investigate and verify a re-render optimization?

Common pitfalls / misconceptions

- Assuming every render causes a DOM update or is inherently bad.
- Using unstable keys or nested component definitions and unexpectedly resetting state.
- Assuming `memo` blocks state or context updates.

- Optimizing development console counts instead of measuring user-visible work.

Cheat-sheet recap

- State, props, and context can lead to renders; commits determine host changes.
- Component type, position, and key govern state identity.
- Profile realistic workloads, simplify state, then apply targeted optimizations.

Day 52 — Context + custom hooks

Month 2 — Full-Stack Depth, Week 4 — Advanced React + Full Stack (Week 8 overall).

Goal: Share appropriate values through a component tree and extract reusable stateful logic without confusing context, hooks, and global state.

Context

Context lets a provider supply a value to descendants without passing it through every intermediate component. It is useful for cross-cutting values such as a theme, locale, or a scoped service. Consumers read the closest matching provider. A missing provider returns the context's default value, so choose a default deliberately or throw a clear error in a custom consumer hook when a provider is required.

Context is a transport mechanism, not automatically a state-management system or selector-based store. A changed provider value can update consumers of that context, even when they are memoized. An object literal created on each provider render has a new identity; stabilize or split values when profiling shows avoidable fan-out, but avoid premature memoization. Do not put unrelated high-frequency state in one broad context by default.

Custom hooks

A custom hook is a function beginning with `use` that composes React hooks to share stateful behavior, such as a subscription or form-related logic. It obeys the same rules of hooks: call hooks unconditionally at the top level and include correct effect dependencies. Each call normally gets its own hook state; shared state requires a common provider, external store, or another explicit shared owner.

Keep a hook's contract focused: clarify inputs, returned values, cleanup, loading/error states, and whether it performs side effects. Extracting a hook can improve reuse, but splitting simple local code into many opaque hooks can make data flow harder to follow.

Code example

```
import { createContext, useContext, useState, type ReactNode } from "react";

type CounterContextValue = {
  count: number;
  increment: () => void;
};

const CounterContext = createContext<CounterContextValue | null>(null);
```

```
function useCounter(): CounterContextValue {
  const value = useContext(CounterContext);
  if (value === null) throw new Error("useCounter requires CounterProvider.");
  return value;
}

export function CounterProvider({ children }: { children: ReactNode }) {
  const [count, setCount] = useState(0);
  return (
    <CounterContext.Provider value={{ count, increment: () => setCount(c => c + 1)
}}>
      {children}
    </CounterContext.Provider>
  );
}

export function CounterButton() {
  const { count, increment } = useCounter();
  return <button type="button" onClick={increment}>Count: {count}</button>;
}
```

Place `CounterButton` inside `CounterProvider`. The provider creates a new value object on its renders, which is correct but can trigger consumer updates; optimize or split the context only if that matters for the actual tree.

DSA problem: Merge two sorted lists

Problem: Merge two ascending, acyclic, **disjoint** singly linked lists into one ascending list, retaining duplicates. Reuse the existing nodes.

Approach: Use a dummy head and a tail pointer. Repeatedly link the smaller current node, then append whichever list remains. Save the selected node's next pointer before advancing. Reusing nodes mutates the input lists' links.

Complexity: O(m + n) time and O(1) additional space for lists of lengths `m` and `n`, excluding the output nodes because they are reused.

```
class ListNode {
  constructor(public value: number, public next: ListNode | null = null) {}
}

function mergeLists(left: ListNode | null, right: ListNode | null): ListNode |
null {
  const dummy = new ListNode(0);
  let tail = dummy;
  let a = left;
  let b = right;

  while (a !== null && b !== null) {
    if (a.value <= b.value) {
      const next: ListNode | null = a.next;
```

```
      tail.next = a;
      tail = a;
      a = next;
    } else {
      const next: ListNode | null = b.next;
      tail.next = b;
      tail = b;
      b = next;
    }
  }
  tail.next = a !== null ? a : b;
  return dummy.next;
}

const left = new ListNode(1, new ListNode(3));
const right = new ListNode(2, new ListNode(3));
const merged = mergeLists(left, right);
console.log(merged?.value, merged?.next?.value, merged?.next?.next?.value); // 1 2
3
```

Edge cases: Either input may be empty; duplicates and negative values work. Inputs must be sorted, acyclic, and not share nodes; shared nodes can make a reuse-based merge create a cycle. Allocate new nodes if preserving original links is required.

Likely interview questions

1. What problem does context solve, and what does it not automatically provide?
2. What happens when a consumer has no matching provider?
3. Why can a newly created provider value update many consumers?
4. When might splitting context be more useful than memoizing every consumer?
5. Does calling the same custom hook twice share state?
6. What cleanup and dependency obligations apply to a custom hook?
7. Why does the merge algorithm use a dummy head?
8. What happens if two input lists share nodes and the algorithm reuses them?

Common pitfalls / misconceptions

- Treating context as a free replacement for every state-management need.
- Assuming custom hook calls automatically share one state instance.
- Silently accepting a missing provider when the component cannot function correctly.
- Reusing nodes from overlapping lists without accounting for cycles.

Cheat-sheet recap

- Context distributes a value; an explicit owner controls shared state.
- Custom hooks reuse stateful logic and obey hook ordering and cleanup rules.
- Merge sorted disjoint lists with a dummy head and tail; specify mutation.

Day 53 — Component architecture

Month 2 — Full-Stack Depth, Week 4 — Advanced React + Full Stack (Week 8 overall).

Goal: Organize components around ownership, reusable contracts, and feature boundaries rather than arbitrary layers.

Responsibilities and composition

A component should have a clear purpose and a contract expressed through props, events, and accessible behavior. Separate presentation from data access when it improves reuse or testability, but do not force every small component into “container” and “presentational” files. Compose focused components and pass children or callbacks when that makes variation explicit.

Group code by feature when it helps developers find a feature's UI, API access, types, and tests together. Shared components should capture genuinely common behavior, not every superficial resemblance. A premature universal component with many boolean props can become harder to understand than two small components.

State and data boundaries

Keep state close to where it is used, lift it to the nearest common owner when siblings must coordinate, and use context or a store when the sharing scope justifies it. Distinguish client UI state, form state, and server state: they have different lifetimes and consistency needs. Avoid duplicating the same source of truth in multiple places without a synchronization plan.

For asynchronous features, model loading, empty, success, and error states explicitly. Define ownership of validation, API calls, cancellation, and retry behavior. An accessible component should use semantic HTML, labels, keyboard behavior, and meaningful error messages.

Contracts, testing, and tradeoffs

Use TypeScript to express component inputs, but validate untrusted data at runtime boundaries. Components should not assume that a compile-time interface validates a server response. Keep business rules in an appropriate shared or server-side boundary rather than hiding critical authorization or validation only in the UI.

Test behavior as a user sees it, and add focused tests for complex logic and integration boundaries. Consider render cost, bundle size, and dependency direction, but measure before introducing a large architecture. A senior design discussion explains why a boundary exists and what it costs.

Code example

```
import { useState } from "react";

type Item = { id: string; title: string };

function ItemList({ items, onRemove }: {
  items: Item[];
  onRemove: (id: string) => void;
}) {
  if (items.length === 0) return <p>No items.</p>;
```

```
  return (
    <ul>
      {items.map(item => (
        <li key={item.id}>
          {item.title}
          <button type="button" onClick={() => onRemove(item.id)}>
            Remove {item.title}
          </button>
        </li>
      ))}
    </ul>
  );
}

export function ItemPanel() {
  const [items, setItems] = useState<Item[]>([
    { id: "1", title: "Review" },
    { id: "2", title: "Test" },
  ]);
  return (
    <ItemList
      items={items}
      onRemove={id => setItems(current => current.filter(item => item.id !== id))}
    />
  );
}
```

The owner manages state and the list receives a clear contract. This is local UI state only; it does not persist removals or handle an API.

Likely interview questions

1. How do you decide where state should live?
2. When is a presentation/data-access split useful, and when is it unnecessary?
3. What distinguishes UI, form, and server state?
4. How would you organize a feature so its API, types, UI, and tests remain discoverable?
5. What signals that a reusable component has too many responsibilities?
6. How should a component represent loading, empty, and error states?
7. Why must security-sensitive validation remain on the server?
8. How would you test the behavior of a reusable component?

Common pitfalls / misconceptions

- Creating global state for data used in one small subtree.
- Duplicating server data into several independent stores without a synchronization strategy.
- Building a highly configurable shared component before a real shared need exists.
- Treating TypeScript props as runtime validation or authorization.

Cheat-sheet recap

- Design around clear responsibilities, contracts, accessibility, and data ownership.

- Keep state as local as practical and avoid duplicate sources of truth.
- Reuse components and hooks where actual variation justifies them.

Day 54 — React + ASP.NET Core integration

Month 2 — Full-Stack Depth, Week 4 — Advanced React + Full Stack (Week 8 overall).

Goal: Trace data from browser to API and back, including HTTP contracts, authentication, validation, and deployment boundaries.

Request and response contract

A React client sends HTTP requests to ASP.NET Core endpoints and interprets status codes and response bodies. Agree on routes, methods, JSON property names, error shapes, pagination, and versioning. `fetch` resolves for HTTP error statuses, so inspect `response.ok` or the specific status before parsing a success body. A TypeScript type declaration is not runtime validation; treat external JSON as `unknown` until checked.

Model loading, empty, success, and error states separately. Cancel or ignore obsolete reads when a component changes or unmounts. For mutations, a timeout does not prove the server rolled back; define retry and idempotency behavior. Use stable IDs and an explicit strategy for refreshing or invalidating cached server data.

Authentication, authorization, and browser boundaries

The API must authenticate callers and enforce endpoint and resource-level authorization. Hiding a button in React improves UX but is not authorization. For bearer-token authentication, send the access token using the configured scheme over HTTPS; avoid logging it or placing it in URLs. Cookie-based authentication has different CSRF and browser credential concerns.

If frontend and API are on different origins, the API needs an appropriate CORS policy for browser access; same-origin production hosting or a development proxy can avoid cross-origin browser requests. CORS is not authentication. With a reverse proxy, trust forwarded headers only from configured proxies and verify the external scheme and host.

Reliability and observability

Pass cancellation through the request where useful, use timeouts and bounded retries only when safe, and expose actionable errors without leaking internal details. Correlation IDs and distributed tracing can connect a frontend request to backend logs. Check API version compatibility, serialization of dates and decimal amounts, and the behavior of deployment environments before assuming a local integration will work in production.

Code example

This TypeScript function expects an authenticated API to return an array of objects with string `id` and `title` fields. The caller supplies any required authentication through the application's configured mechanism:

```
type Order = { id: string; title: string };

function isOrder(value: unknown): value is Order {
  return typeof value === "object" && value !== null
```

```
    && "id" in value && typeof value.id === "string"
    && "title" in value && typeof value.title === "string";
}

async function loadOrders(signal: AbortSignal): Promise<Order[]> {
  const response = await fetch("/api/orders", { signal });
  if (!response.ok) {
    throw new Error(`Could not load orders: HTTP ${response.status}`);
  }
  const body: unknown = await response.json();
  if (!Array.isArray(body) || !body.every(isOrder)) {
    throw new Error("Invalid order response.");
  }
  return body;
}
```

An application component should await this function, show or propagate its error, and abort or ignore an obsolete request. The API must independently verify the caller's access to each returned order.

Likely interview questions

1. How do you define and validate a frontend/backend JSON contract?
2. Why does `fetch` require an explicit HTTP status check?
3. How do you handle 401, 403, validation errors, and unexpected failures in the UI?
4. Why does hiding a UI control not secure an API?
5. When is CORS required, and how does a development proxy change the browser's origin?
6. How would you prevent an obsolete read from overwriting newer UI state?
7. Why can retrying a timed-out create operation produce duplicates?
8. How would you trace a slow request across the browser and ASP.NET Core service?

Common pitfalls / misconceptions

- Trusting TypeScript types to validate arbitrary response JSON.
- Treating CORS as authorization or relying on UI controls for access control.
- Assuming an aborted or timed-out mutation did not reach the server.
- Ignoring deployment-specific URLs, proxy configuration, and error contracts.

Cheat-sheet recap

- Agree on HTTP and JSON contracts; validate untrusted responses.
- Authenticate and authorize on the server; handle browser origin and credential rules separately.
- Model async states, cancellation, retries, and observability explicitly.

Day 55 — Frontend performance

Month 2 — Full-Stack Depth, Week 4 — Advanced React + Full Stack (Week 8 overall).

Goal: Diagnose user-visible frontend bottlenecks and choose optimizations based on field and lab evidence.

Measure the user experience

Start with the affected route, device, network, and interaction. Field metrics describe actual user experience; lab profiles help reproduce and diagnose it. Core Web Vitals include Largest Contentful Paint (loading experience), Interaction to Next Paint (interaction responsiveness), and Cumulative Layout Shift (visual stability). Segment metrics by device, network, route, and release rather than relying only on an average.

Use browser performance tools to inspect network waterfalls, main-thread tasks, script execution, layout, paint, and memory. React DevTools Profiler helps locate expensive React commits. Correlate frontend timing with backend latency: a slow API and a large JavaScript bundle require different fixes.

Loading and runtime work

Reduce unnecessary JavaScript through route-level code splitting, removing unused dependencies, and loading optional features on demand. Optimize images and fonts, use appropriate caching and compression, and avoid unnecessary request waterfalls. Preloading too much can compete with critical resources; verify its effect.

For runtime performance, keep expensive work off critical interactions where practical, avoid unnecessary effects and repeated computations, and consider virtualization for genuinely large lists. Virtualization can affect accessibility, focus management, variable-height content, and browser search, so test those behaviors. Memoization helps only where repeated work or stable identities matter.

Verify and guard against regressions

Establish a baseline, change one plausible cause, and compare realistic before/after runs and field data. Track user-visible latency, bundle size, network requests, render cost, and errors; a faster metric that breaks functionality or accessibility is not a successful optimization. Performance budgets and automated checks can detect regressions, but should reflect actual user priorities.

Code example

This browser example observes long tasks when supported and returns a cleanup function. A long task is a diagnostic signal, not by itself proof that React caused the delay.

```
function observeLongTasks(): () => void {
  if (!("PerformanceObserver" in window)
      || !PerformanceObserver.supportedEntryTypes.includes("longtask")) {
    console.info("Long-task observation is not supported in this browser.");
    return () => {};
  }

  const observer = new PerformanceObserver(list => {
    for (const entry of list.getEntries()) {
      console.log(`Long task: ${entry.duration.toFixed(1)} ms`);
    }
  });
  observer.observe({ type: "longtask", buffered: true });
  return () => observer.disconnect();
}
```

```
const stopObserving = observeLongTasks();
// Call stopObserving() when monitoring is no longer needed.
```

Support and detail vary by browser. Avoid collecting sensitive information in performance telemetry, and verify an improvement with representative users and conditions.

Likely interview questions

1. How do field metrics and lab profiles complement each other?
2. What do LCP, INP, and CLS measure?
3. How would you distinguish API latency from main-thread rendering work?
4. When does code splitting help, and what tradeoffs can it introduce?
5. What should you check before virtualizing a list?
6. Why can development-mode render counts be misleading?
7. How would you verify that a memoization change helped?
8. What correctness and accessibility checks accompany a performance optimization?

Common pitfalls / misconceptions

- Optimizing render counts without measuring user-visible delay.
- Assuming a slow screen must be caused by React rather than network or backend work.
- Virtualizing without testing keyboard and assistive-technology behavior.
- Improving one lab metric while breaking functionality or worsening real-user experience.

Cheat-sheet recap

- Measure real users and reproduce the affected interaction.
- Separate network, JavaScript, rendering, layout, and backend costs.
- Optimize the measured bottleneck and verify performance, correctness, and accessibility.

Day 56 — Build/review a small full-stack feature end to end

Month 2 — Full-Stack Depth, Week 4 — Advanced React + Full Stack (Week 8 overall).

Goal: Trace a feature from a React form through an ASP.NET Core API and back, then identify the work required for production.

Feature design

The illustrative feature lists and creates work items. The browser validates the title for immediate feedback; the API validates it again and returns explicit HTTP outcomes. The client handles loading, empty, success, and failure states and validates the response shape. A development proxy sends `/api` requests to ASP.NET Core.

The server stores items **in memory only**, so data is lost on restart and differs across application instances. It has no authentication, authorization, durable persistence, concurrency/version policy, idempotency keys, or production observability. This is a runnable integration exercise, not a production-ready feature or a claim about Sai's work.

Worked example: ASP.NET Core API

Create a .NET web project with `dotnet new web -n WorkApi`, replace its `Program.cs` with the following, and run it using `dotnet run -- --urls http://localhost:5055` from the project directory:

```
using System.Collections.Concurrent;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddProblemDetails();
var app = builder.Build();
app.UseExceptionHandler();

var items = new ConcurrentDictionary<Guid, WorkItem>();

app.MapGet("/api/items", () =>
    Results.Ok(items.Values.OrderBy(item => item.Title).ThenBy(item =>
item.Id).ToArray()));

app.MapGet("/api/items/{id:guid}", (Guid id) =>
    items.TryGetValue(id, out var item) ? Results.Ok(item) : Results.NotFound());

app.MapPost("/api/items", (CreateItem request) =>
{
    string title = request.Title?.Trim() ?? "";
    if (title.Length is < 1 or > 100)
        return Results.ValidationProblem(new Dictionary<string, string[]>
        {
            ["title"] = new[] { "Title must contain 1 to 100 characters." }
        });

    var item = new WorkItem(Guid.NewGuid(), title);
    if (!items.TryAdd(item.Id, item))
        throw new InvalidOperationException("Could not allocate a unique item
identifier.");
    return Results.Created($"/api/items/{item.Id}", item);
});

app.Run();

public sealed record CreateItem(string? Title);
public sealed record WorkItem(Guid Id, string Title);
```

The dictionary supports concurrent access within this process. It does not provide durable storage, a transactional workflow, or cross-instance coordination. The generated `Location` points to the GET-by-ID route.

Worked example: React client

In a React + TypeScript Vite project, configure the development proxy in `vite.config.ts`:

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
```

```
export default defineConfig({
  plugins: [react()],
  server: {
    proxy: { "/api": { target: "http://localhost:5055", changeOrigin: true } },
  },
});
```

Replace the application's `App.tsx` with:

```
import { useEffect, useState, type FormEvent } from "react";

type WorkItem = { id: string; title: string };
type ListState =
  | { kind: "loading" }
  | { kind: "ready"; items: WorkItem[] }
  | { kind: "error"; message: string };

function isWorkItem(value: unknown): value is WorkItem {
  return typeof value === "object" && value !== null
    && "id" in value && typeof value.id === "string"
    && "title" in value && typeof value.title === "string";
}

export default function App() {
  const [state, setState] = useState<ListState>({ kind: "loading" });
  const [title, setTitle] = useState("");
  const [saving, setSaving] = useState(false);
  const [formError, setFormError] = useState<string | null>(null);

  useEffect(() => {
    const controller = new AbortController();
    async function load() {
      try {
        const response = await fetch("/api/items", { signal: controller.signal });
        if (!response.ok) throw new Error(`HTTP ${response.status}`);
        const body: unknown = await response.json();
        if (!Array.isArray(body) || !body.every(isWorkItem))
          throw new Error("Invalid item list response.");
        if (!controller.signal.aborted) setState({ kind: "ready", items: body });
      } catch (error: unknown) {
        if (!controller.signal.aborted) {
          setState({
            kind: "error",
            message: error instanceof Error ? error.message : "Could not load
items.",
          });
        }
      }
    }
    void load();
    return () => controller.abort();
```

```
  }, []);

  async function createItem(event: FormEvent<HTMLFormElement>) {
    event.preventDefault();
    if (state.kind !== "ready" || saving) return;
    const trimmed = title.trim();
    if (trimmed.length < 1 || trimmed.length > 100) {
      setFormError("Title must contain 1 to 100 characters.");
      return;
    }

    setSaving(true);
    setFormError(null);
    try {
      const response = await fetch("/api/items", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ title: trimmed }),
      });
      if (!response.ok) throw new Error(`HTTP ${response.status}`);
      const body: unknown = await response.json();
      if (!isWorkItem(body)) throw new Error("Invalid created-item response.");
      setState(current => current.kind === "ready"
        ? { kind: "ready", items: [...current.items, body] }
        : current);
      setTitle("");
    } catch (error: unknown) {
      const detail = error instanceof Error ? error.message : "Request failed.";
      setFormError(`${detail} The item may have been created; reload before
retrying.`);
    } finally {
      setSaving(false);
    }
  }

  return (
    <main>
      <h1>Work items</h1>
      {state.kind === "loading" && <p>Loading...</p>}
      {state.kind === "error" && <p role="alert">Could not load items:
{state.message}</p>}
      {state.kind === "ready" && (
        state.items.length === 0
          ? <p>No items yet.</p>
          : <ul>{state.items.map(item => <li key={item.id}>{item.title}</li>)}
</ul>
      )}
      <form onSubmit={createItem}>
        <label htmlFor="title">Title</label>
        <input id="title" value={title} onChange={event =>
setTitle(event.target.value)} />
        {formError && <p role="alert">{formError}</p>}
        <button type="submit" disabled={saving || state.kind !== "ready"}>
          {saving ? "Saving..." : "Create"}
```

```
          </button>
        </form>
      </main>
    );
  }
```

Run the Vite development server and open its printed URL. The proxy is a development configuration; production routing, HTTPS, and security must be configured separately. The client appends a created item locally, so its displayed order may differ from the API's sorted list until reload.

End-to-end verification and production review

1. Load the page with the API running: an empty list should be visible initially.
2. Submit a valid title: the API should return 201 and the item should appear; reload and verify it remains while the API process is still running.
3. Submit an empty or overlong title: the client should show validation feedback. Call the API directly with invalid input to verify server-side 400 validation independently.
4. Stop the API and reload: the client should show a loading failure rather than an empty success state.
5. Restart the API: the in-memory data disappears, demonstrating the persistence limitation. Test malformed responses, concurrent requests, and ambiguous mutation failures in a proper test environment.

For actual deployment, add durable persistence and migrations, identity and resource-level authorization, secure transport, structured diagnostics, automated integration tests, deployment configuration, and a deliberate concurrency/idempotency design. Add CORS only when the production browser origins require it. A financial feature also needs explicit currency, precision, auditing, and domain rules.

Real-world tie-in: your full-stack description

The roadmap says you should now be able to describe yourself as a **C#/.NET backend + React/TypeScript frontend full-stack developer**. Ground that description in work you actually did: identify a real feature, your backend and frontend responsibilities, the integration contract, a meaningful technical decision, and how you verified it. Distinguish personal contributions from team work and planned work from shipped work.

Model answer shape: “I work across [actual C#/.NET backend responsibilities] and [actual React/TypeScript frontend responsibilities]. On [real feature], I contributed [specific work] across [actual API and UI boundary]. We chose [real approach] because [constraint or tradeoff], and verified it using [actual tests or evidence]. One limitation or improvement was [honest detail].”

Likely interview questions

1. Trace the create operation from form input to API response and UI update.
2. Why does the server validate input even when the client already validated it?
3. How does the development proxy affect browser origins, and what changes in production?
4. What failure states can occur during the initial read and during the create request?
5. Why can a timed-out create request require an idempotency strategy?
6. What limitations does in-memory server storage introduce?
7. How would you add authentication and resource-level authorization to this feature?
8. Describe a real full-stack feature and distinguish your contribution from the team's.

Common pitfalls / misconceptions

- Treating a working local integration as production-ready.
- Relying on client validation, a proxy, or hidden controls as server-side security.
- Retrying an ambiguous create without considering duplicate side effects.
- Claiming this illustrative feature is an actual shipped project or inventing personal contributions.

Cheat-sheet recap

- Trace the contract end to end: UI → HTTP → validation → application behavior → response → UI.
- Test success, validation, failure, restart, and authorization boundaries.
- Add persistence, security, concurrency, observability, and deployment controls for real use.
- Describe genuine C#/.NET and React/TypeScript contributions with evidence and tradeoffs.